

Molecular dynamics: Lennard-Jones particles

a. Initial positions of particles on a cubic lattice

2 important things to notice:

- If particles start closer than σ , the repulsion gives huge forces and the simulation blows up.
- If they start too far apart, they would behave more like a gas rather than a liquid.
- The minimum of the LJ potential is at $r = 2^{1/6} \sigma \approx 1.12 \sigma$. There, $F = -dV/dr = 0$, so neighbours don't push or pull on each other at the start. Therefore, it is a reasonable initial distance.

```
In [1]: import numpy as np  
import matplotlib.pyplot as plt
```

```

In [ ]: # Lennard-Jones potential:  $V_{LJ}(r) = 4\epsilon\sigma^{12} \left[ \left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^6 \right]$ 
#  $V = 0$  at  $r = \sigma$ , minimum  $V = -\epsilon$  at  $r = 2^{1/6}\sigma$ 
# At the minimum  $dV/dr = 0$ , so the force between two neighbours is zero
# "The constants  $\sigma$  and  $\epsilon$  both can be set to unity in the code" so  $\sigma = 1$  and  $\epsilon = 1$ 

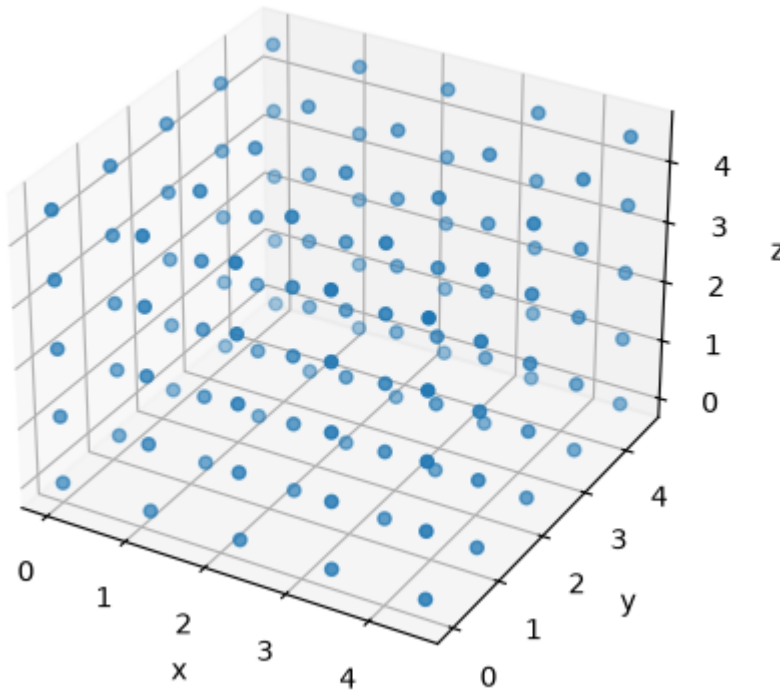
# I will spread  $n$  particles per side, so  $N = n^3$  particles in total
n = 5
N = n**3

# Cubic lattice: particle positions are  $a(i, j, k)$  with  $i, j, k = 0, \dots, n-1$ 
#  $a = 2^{1/6}$  (minimum of the LJ potential)
a = 2**(1/6)

# Initial positions: all combinations of  $i, j, k$ , multiplied by  $a$ 
idx = np.arange(n) #lattice indices along one side
i, j, k = np.meshgrid(idx, idx, idx, indexing='ij') #generate every
combination of  $(i, j, k)$ , indexing is to make sure each index goes to
correct axis
#  $i, j, k$  are now 3D arrays (each store the index of each point of its axis)
# I will flatten each into a 1D list of length  $N$  using .ravel
# After that, I will stack them next to each other as columns using
np.column_stack
# Then I have a table that I can plot with
pos = a*np.column_stack([i.ravel(), j.ravel(), k.ravel()]) #shape (N, 3)

# Plot in 3d
ax = plt.figure().add_subplot(projection='3d')
ax.scatter(pos[:, 0], pos[:, 1], pos[:, 2]) #go through every row of the
columns of pos
ax.set_xlabel("x"); ax.set_ylabel("y"); ax.set_zlabel("z")
plt.show()

```



b. Initial velocities

Note:

- In Maxwell distribution, each velocity component is Gaussian with mean 0 and variance $k_B T/m$ (Eq. 8.58 in the book).
- Same as section 8.4 in the book, I use ϵ as the unit of energy, ϵ/k_B as the unit of temperature and σ as the unit of length.
- I also set $m = 1$ for simplicity ($a = F$, just in case you say that I don't explain enough, this is the Newton's second law of motion $F = dp/dt$, and $p = mv$ (this is the momentum with m is the inertial mass, which has been proven to be equal to the gravitational mass, just in case you are having a bad day, again, of course), so we have $F = m*(dv/dt) + v*(dm/dt)$. But in this case we assume the mass does not so we have $dm/dt = 0$, and then as I said, we set $m = 1$ for simplicity so we get $a = F$).
- Then velocity is in units of $\sqrt{\epsilon/m}$ (I know it's $v = \sqrt{2E/m}$, but I am talking about units, just in case you are having a bad day), and the variance is just T (I hope you can still get this even in a bad day).

```
In [ ]: # Maxwell distribution for one velocity component(x for e.g.):  $W(v_x) \propto \exp(-m*v_x^2 / (2*k_B*T))$ 
# This is a Gaussian with mean 0 and variance  $k_B*T/m$  (just in case you are having a bad day and missed the above markdown cell)
# x, y and z are independent, so each component is drawn separately (just in case you are having a bad day and have a lot of questions than normal)
# Each component has average kinetic energy  $1/2 k_B*T$  (need explanation? It's the equipartition theorem from statistical physics) so the average kinetic energy per particle is  $3/2 k_B*T$  (we have x, y, z just in case you are having a bad day)

# Reduced units:  $\sigma = \epsilon = m = k_B = 1$ 
# So we have length in  $\sigma$ , energy in  $\epsilon$ , temperature in  $\epsilon/k_B$ , velocity in  $\sqrt{\epsilon/m}$ , time in  $\sigma*\sqrt{m/\epsilon}$ 
# In these units the variance  $k_B*T_{real}/m$  is just  $T$  (just in case, again)

# Temperature in reduced units ( $T = k_B*T/\epsilon$ )
T = 1.0

# Each component vx, vy, vz is Gaussian with mean = 0 and standard deviation =  $\sqrt{T}$ 
# Standard deviation =  $\sqrt{\text{variance}}$  and the variance is  $T$  (just in case you don't know)
# rng: random number generator
# rng.normal(mean, std, size) draws Gaussian random numbers we generated
# size = (N, 3): 1 row for each particle, columns are vx, vy, vz (just in case)
rng = np.random.default_rng()
velocity = rng.normal(0, np.sqrt(T), size=(N, 3))
```

c. Center of mass velocity

Note:

- All particles have the same mass, so the center of mass velocity is just the average velocity.
- Therefore, we just need to subtract it from every particle for the system to not drift.

```
In [ ]: # Center of mass velocity = average of all velocities (equal masses)
v_cm = velocity.mean(axis=0)
print(v_cm)

# Subtract it from every particle
velocity = velocity - v_cm
print(velocity.mean(axis=0)) # notice the e^-17 (when I ran this) at the end of each number for row 2 just in case you are having a bad day

[ 0.2347382 -0.0179655  0.05359117]
[-4.61852778e-17  8.88178420e-18 -3.10862447e-17]
```

Before the correction the center of mass velocity is small but not zero, because N is finite. After subtracting it, based on the result we get, it is practically zero ($\sim 1e-17$). It is not completely zero

because of numerical error.